

Your secret files are mine:

Bug finding and exploit techniques on file transfer app of all top
Android vendors

Zhang Xiangqian

Liu Huiming

腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

About us

- **Tencent**
 - Largest social media and entertainment company in China
- **Tencent Security Xuanwu Lab**
 - Applied and real world security research
- **Members of Advanced Security Team**
 - Zhang Xiangqian(@h3rb0)
 - Huiming Liu (@liuhm09)



Contents

- Introduction
- Attack Surfaces and Previous Works
- Vulnerabilities & Exploit & Demo
- Mitigation
- Conclusion



Introduction

- Nearby Transmission Technologies
 - Bluetooth, Android beam, WiFi hotspot, WiFi Direct
- Nearby Sharing on Android
 - Vendor's App
 - xxxx Share, xxxx Drop ...
 - Third Party App
 - SHAREit, Xender, AirDroid, Send Anywhere

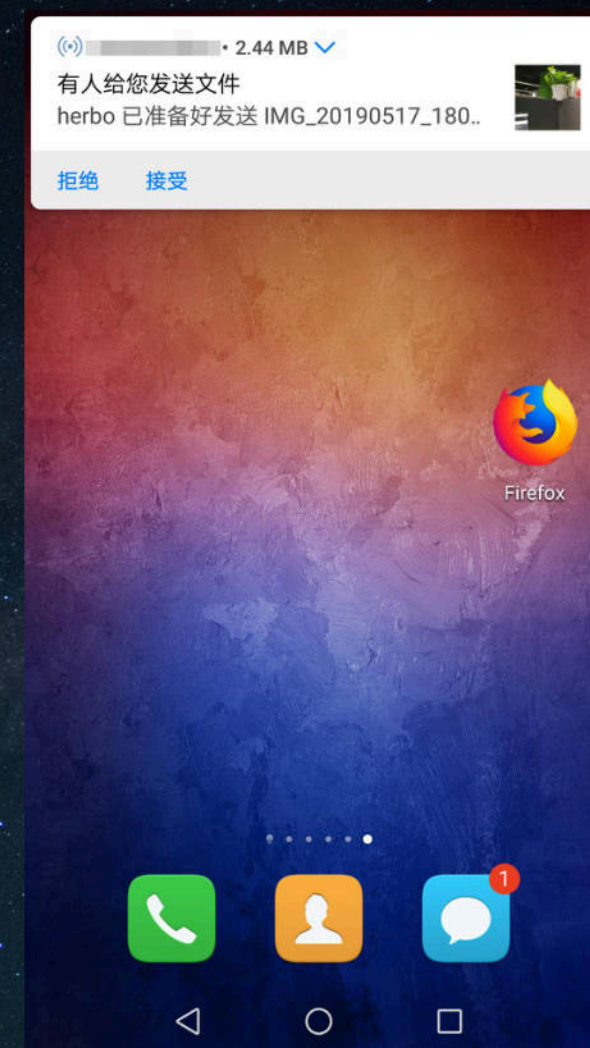
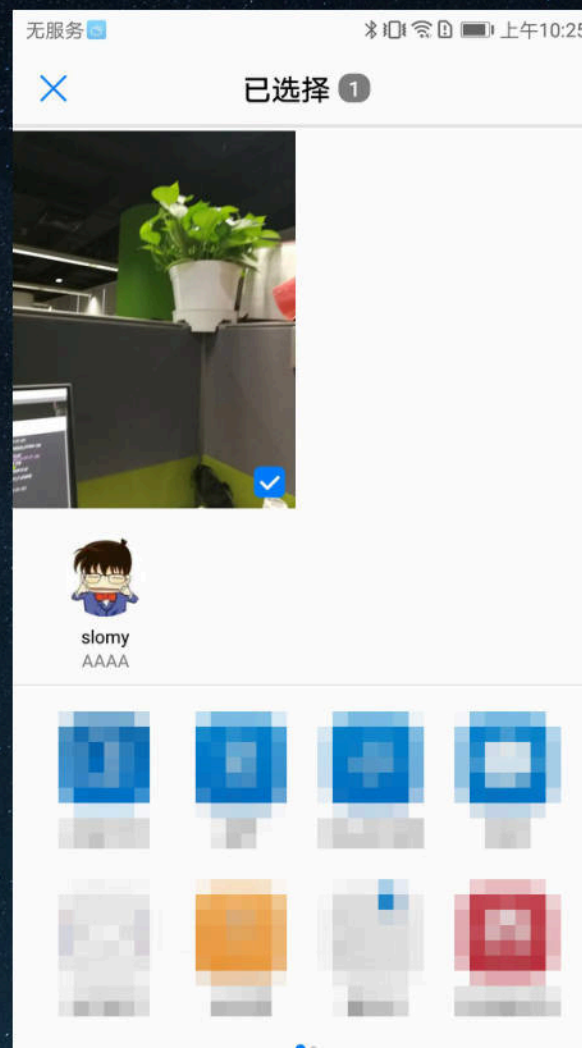
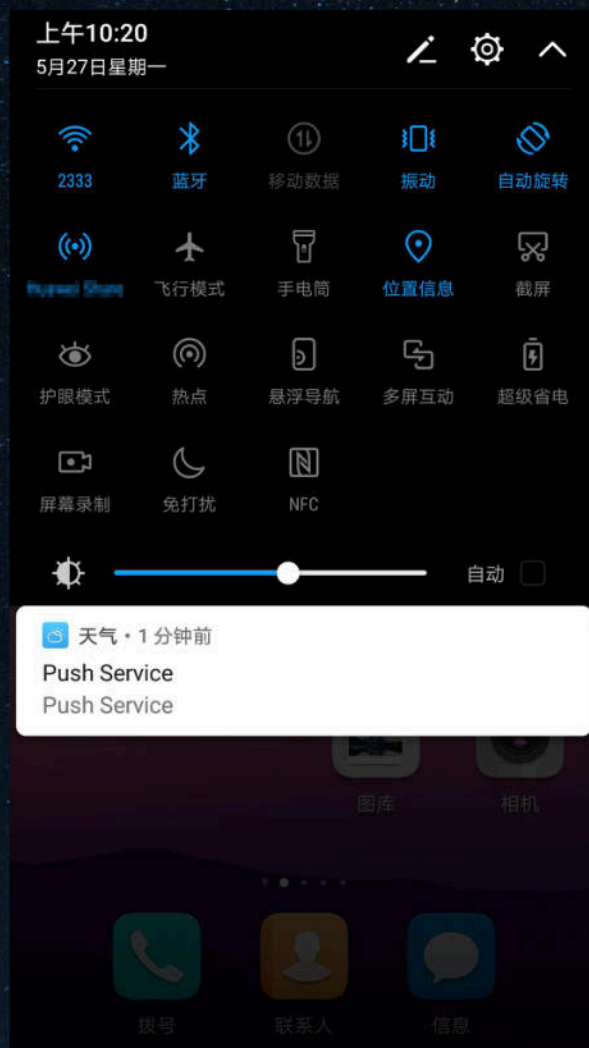


Motivation

- Nearby file transfer
 - Bluetooth/NFC(Android Beam)
 - Not user friendly enough
 - Not fast enough
 - APPs based on Wi-Fi related Technology
- Nearly all top Android Vendors have their Nearby file transfer APPs
 - 2018 shipment: nearly a **billion** android devices (IDC)
- Third-party file transfer Apps
 - Top 10 Apps: nearly a **billion** users
- **But are those Apps safe?**

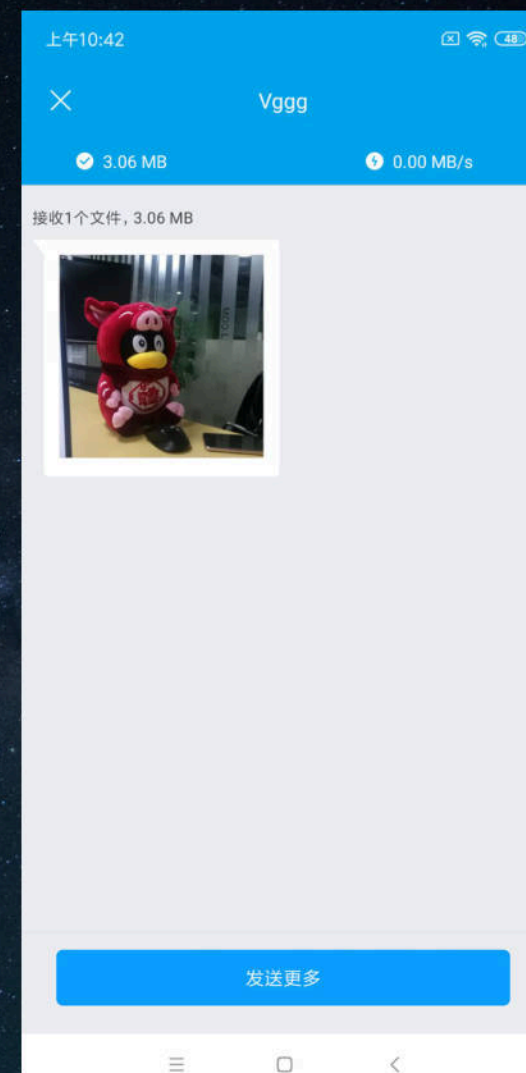
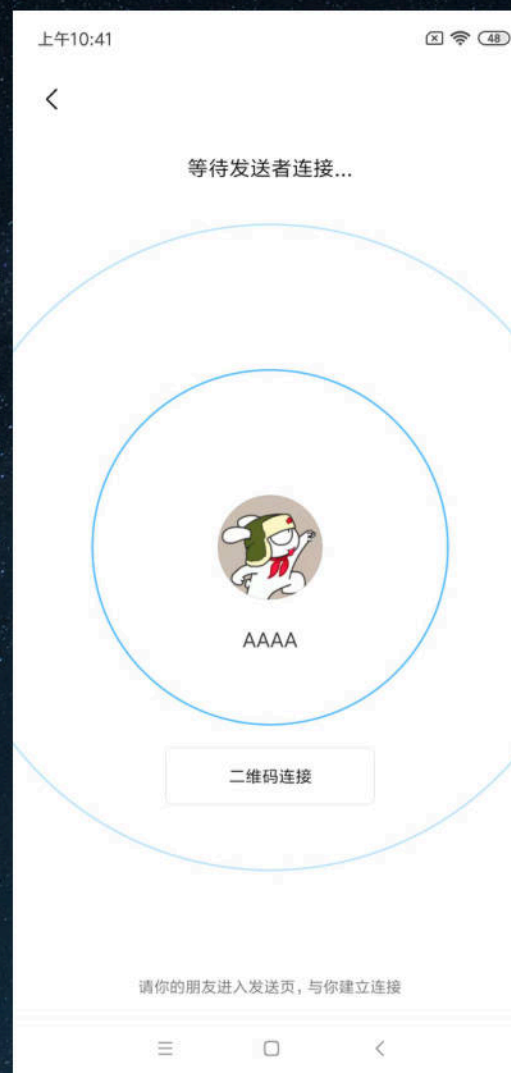
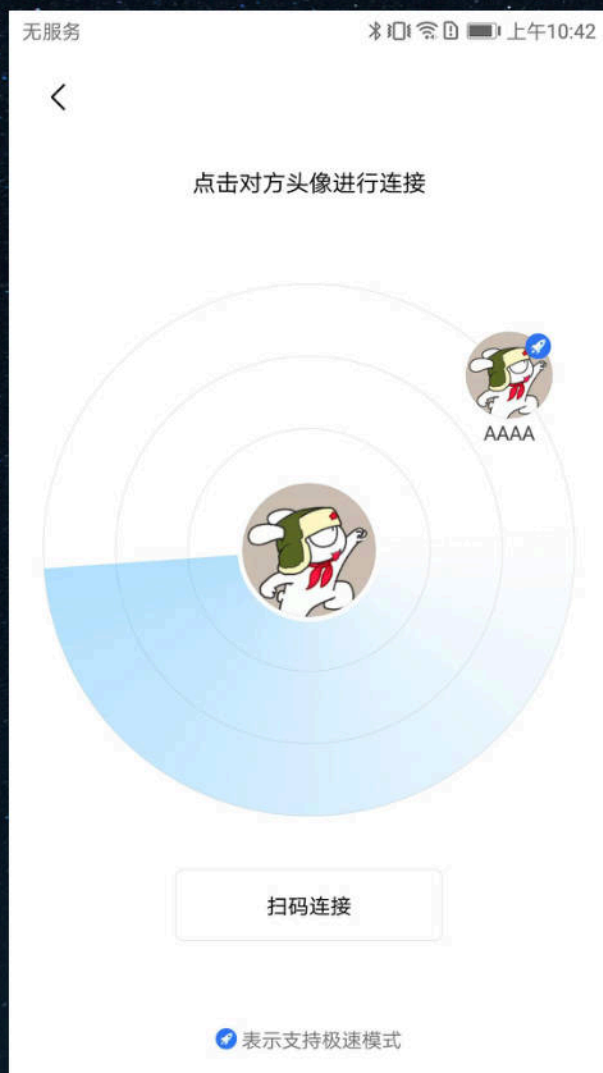


Android Sharing Apps





Android Sharing Apps





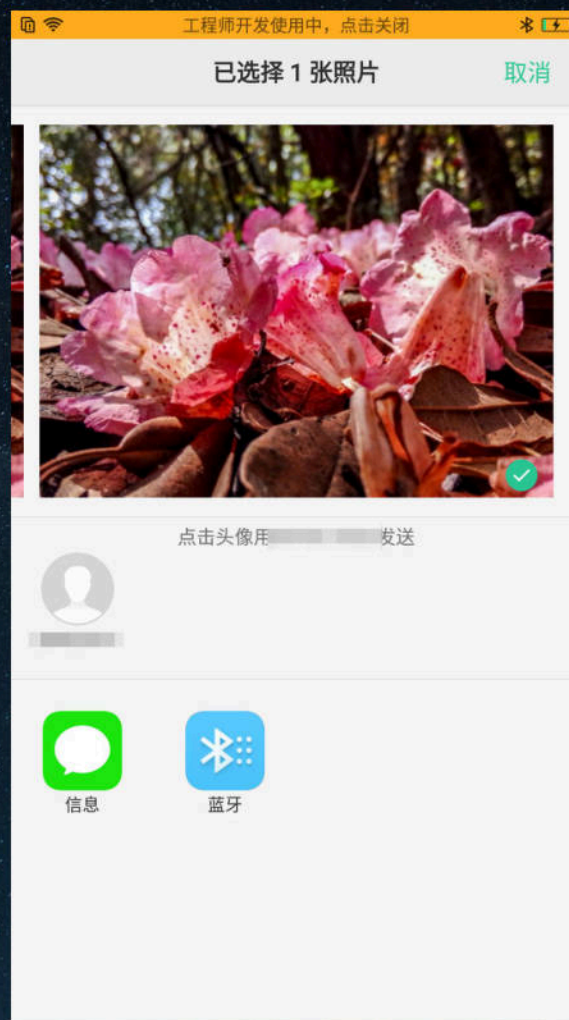
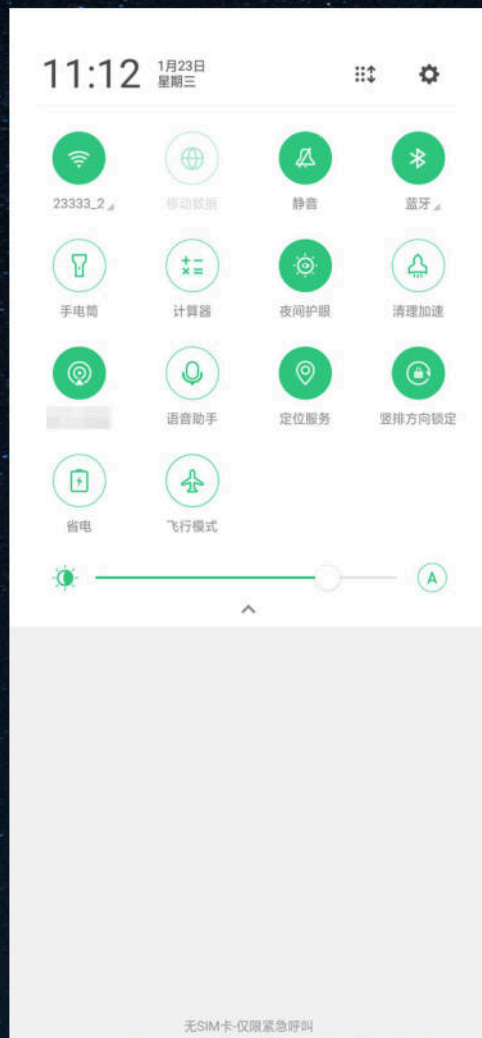
Android Sharing Apps

- Scan The QR code to connect
- Shake at the same time
- ...



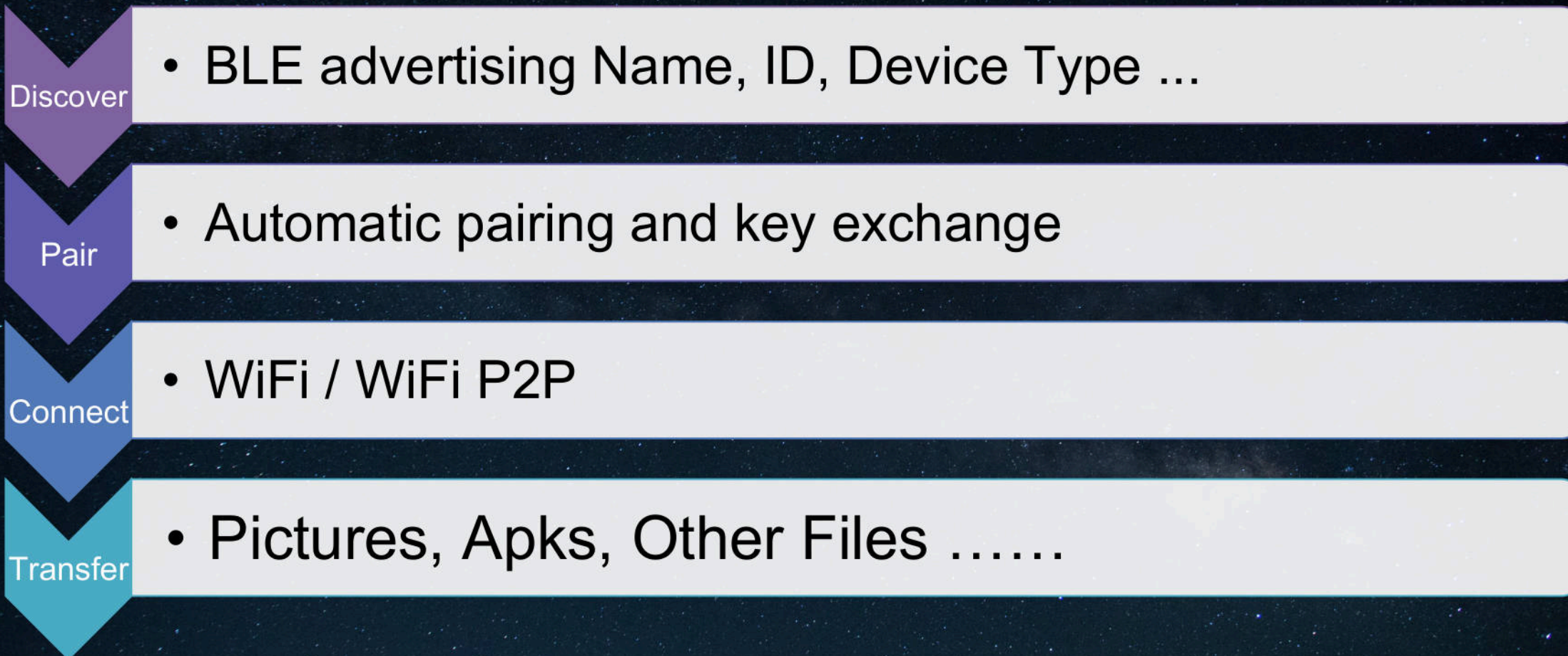
腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Android Sharing Apps





Sharing Process





Attack surface

- Adversary:
 - Attackers can be sniffer and sender of BT and Wi-Fi packets
- Damage:
 - Attackers can get the transferred files
 - Others: Traffic hijacking, Info leaking, RCE...
- Attack Surfaces:
 - Link Establishing
 - Secure Transmission
 - Device / ID Spoofing
 - MITM
 - Others(Web Server)



Link Establishing

- Connect automatically
 - Attackers may join into the network without users' permission.
- Networks' key negotiating
 - Attackers may obtain the established networks' passwords.



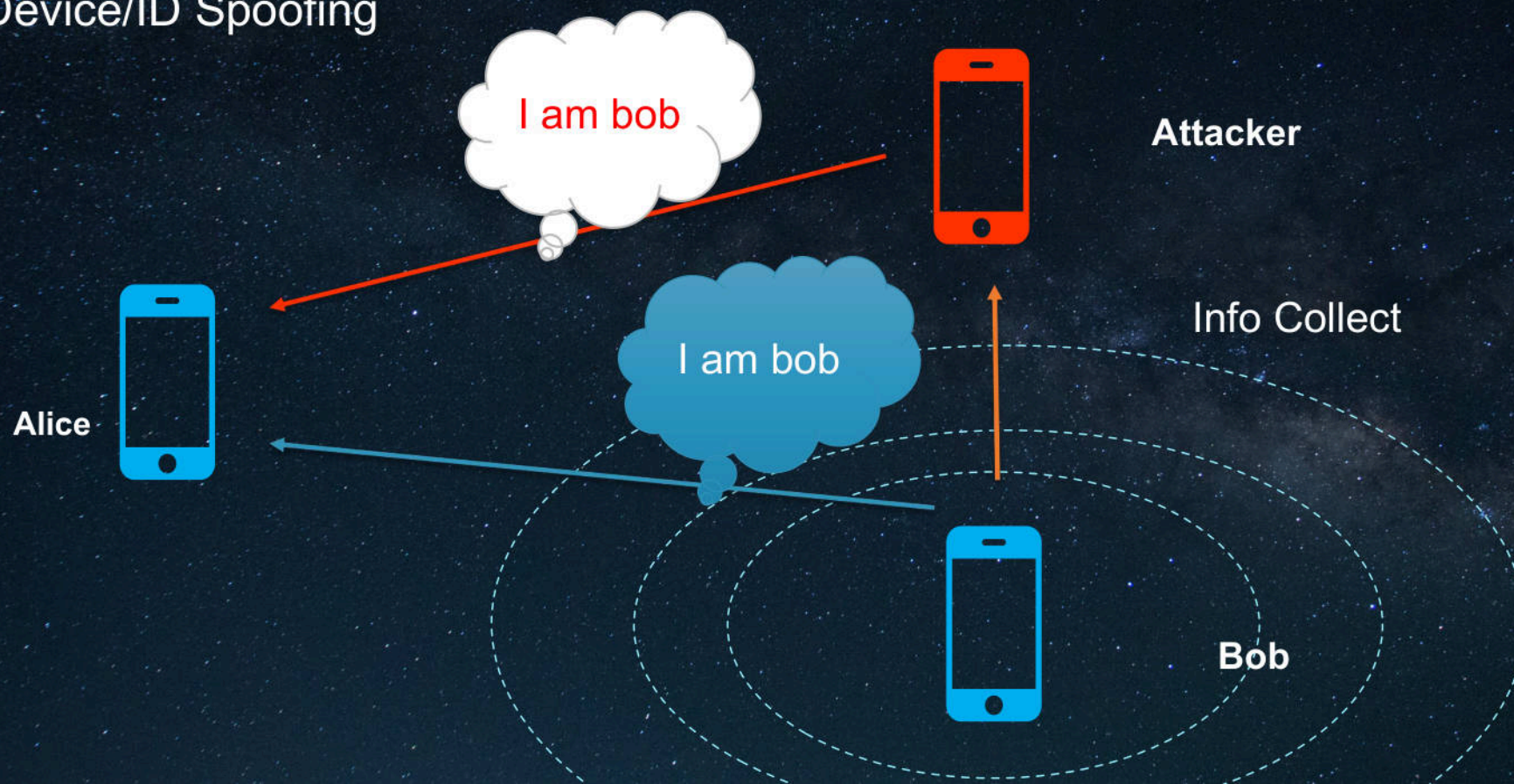
Secure transmission

- No Encryption
 - Get the file directly
- Unsafe key exchange
 - Recover the real file from the encrypted traffic



Device / ID Spoofing

- Authenticate the real devices / users
 - Device/ID Spoofing





Man-in-the-middle

- Bluetooth MITM attack
- WiFi/WiFi P2P MITM attack
- Two Level
 - Users may feel something strange
 - The users can't feel anything (✓)



Others

- Web Server
 - Directory traversal
- Other information disclosure
- Android components related



Previous researches

- ZeroConf on OSX / iOS
 - Discovering and Exploiting Novel Security Vulnerabilities in Apple ZeroConf -- Xiaolong Bai & Luyi Xing (S&P 2016)
- Reverse Google Nearby Connection API
 - Nearby Threats: Reversing, Analyzing, and Attacking Google's 'Nearby Connections' on Android – Antonioli, Daniele, etc. (NDSS '19)



Our Research

- First comprehensive research about Android Nearby Sharing Apps
- Analyzed many related Apps including Pre-installed / Third Party Apps
- Found a lot vulnerabilities in various Apps
- Arrange them in several common attack methods

Vulnerabilities



Vulnerabilities' categories

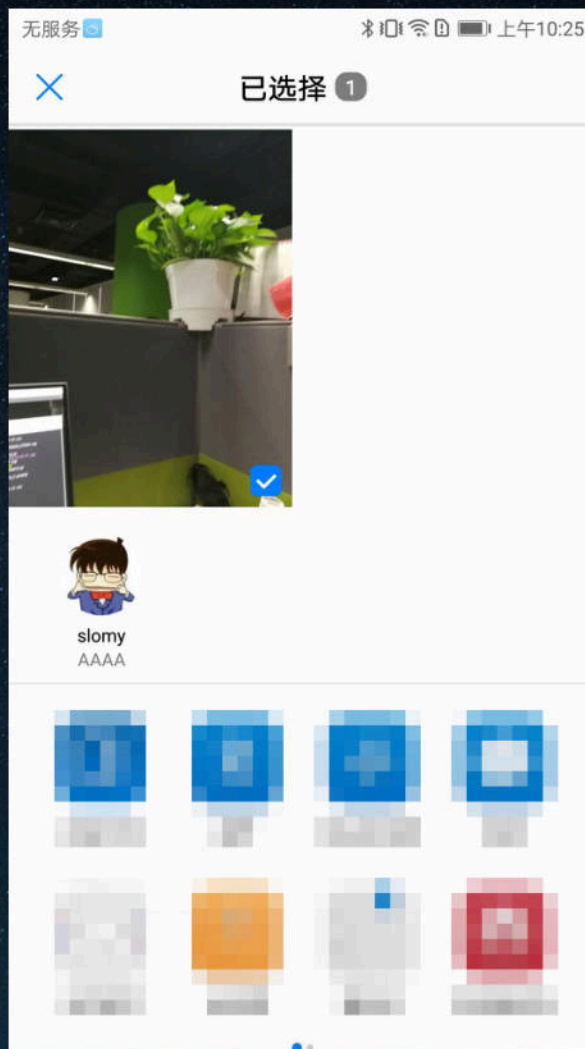
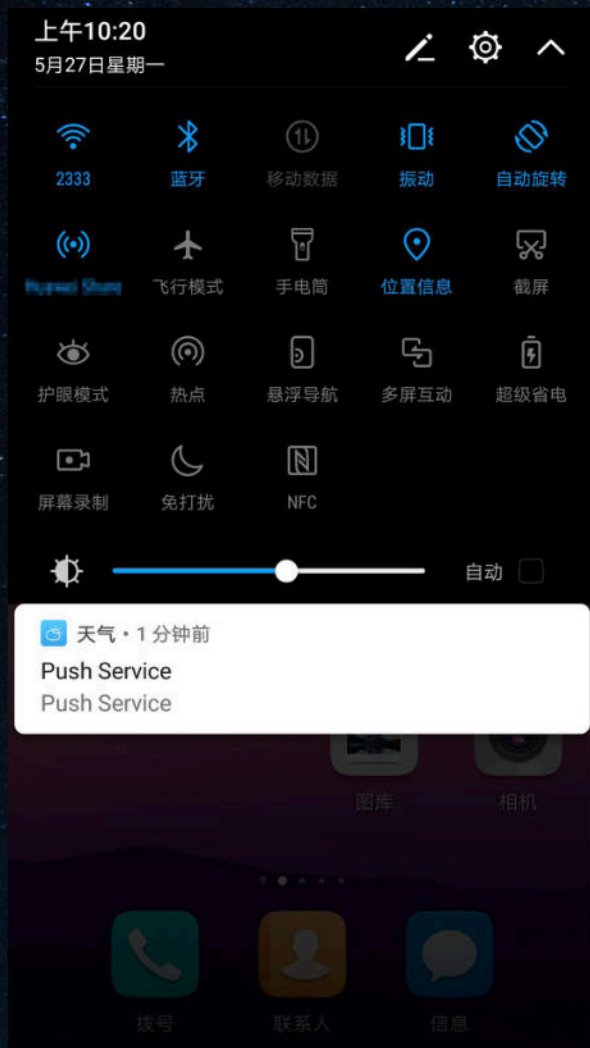
- Sniffer attack related vulnerabilities
- Man-in-the-middle attack related vulnerabilities
- Logic vulnerabilities
- Other vulnerabilities

Sniffer attack

Sniffer attack



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB





BLE Sniffer

- Ubertooth 、 CC2540、 nRF51



BLE Advertising



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Time	Source	Destination	Type	Length
6229	119.380240	62:76:43:3b:17:9a	<broadcast>	BLE ADV
6333	120.628977	41:9c:b0:b1:59:7b	<broadcast>	BLE ADV
6338	120.693979	6f:a4:95:57:2d:9c	<broadcast>	BLE ADV
6343	120.710227	41:9c:b0:b1:59:7b	<broadcast>	BLE ADV
6422	121.784448	5c:14:d2:ac:ff:30	<broadcast>	BLE ADV
6526	123.487728	41:9c:b0:b1:59:7b	<broadcast>	BLE ADV

Frame 6333: 68 bytes on wire (544 bits), 68 bytes captured (544 bits) on interface 0

PPI version 0, 24 bytes

DLT: 147, Payload: btle (Bluetooth Low Energy)

Bluetooth Low Energy

Access Address: 0x8e89bed6

Packet Header

Advertising Address: 41:9c:b0:b1:59:7b (41:9c:b0:b1:59:7b)

Scan Response Data: 1c1635fe0f77485541574549204d6174652032302050726f...

service data: 35fe0f77485541574549204d6174652032302050726f2028...

length: 0x1c

type: Service Data for 16 bit UUID. (0x16)

16 bit uuid: 0xfe35

service data value: 0f77485541574549204d6174652032302050726f20285544...

CRC: 0x43fa0f

Offset	Hex	ASCII
0000	00 00 18 00 93 00 00 00 36 75 0c 00 00 62 09 00	 6u...b..
0010	77 20 e7 47 f1 df 56 00 d6 be 89 8e 44 23 7b 59	w .G..V.D#fY
0020	b1 b0 9c 41 1c 16 35 fe 0f 77 48 55 41 57 45 49	...A..5. .w...ll
0030	20 4d 61 74 65 20 32 30 20 50 72 6f 20 28 55 44	... 32- f (UD
0040	29 43 fa 0f	)C..



slomy
AAAA

Secret data



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

1691 28.094676 [REDACTED]_3d:86:08 ([REDACTED]) 4f:20:62:4f:20:61 () ATT
1695 28.124564 4f:20:62:4f:20:61 () [REDACTED]_3d:86:08 ([REDACTED]) ATT
→ 1696 28.124847 [REDACTED]_3d:86:08 ([REDACTED]) 4f:20:62:4f:20:61 () ATT
← 1706 28.165142 4f:20:62:4f:20:61 () [REDACTED]_3d:86:08 ([REDACTED]) ATT
1707 28.165142 [REDACTED]_3d:86:08 ([REDACTED]) 4f:20:62:4f:20:61 () ATT

Bluetooth HCI H4
Bluetooth HCI ACL Packet
Bluetooth L2CAP Protocol
Bluetooth Attribute Protocol
▶ Opcode: Prepare Write Request (0x16)
▶ Handle: 0x0035 (Unknown: Unknown)
Offset: 54
Value: 574f58
[Response in]
0000 02 40 00 1b 0
0010 58 4d 46 4d 5

251 14.391137 04:4f:4c:d7:39:9e (remote ()) ATT 48 Sent handle value notification, handle: 0x002a
257 14.646274 remote () 04:4f:4c:d7:39:9e (ATT 109 Rcvd write Request, Handle: 0x002a
258 14.667543 04:4f:4c:d7:39:9e (remote ()) ATT 10 Sent write Response

Frame 257: 109 bytes on wire (872 bits), 109 bytes captured (872 bits)
Bluetooth HCI H4
Bluetooth HCI ACL Packet
Bluetooth L2CAP Protocol
Bluetooth Attribute Protocol
Opcode: Write Request (0x12)
Handle: 0x002a
Value: 7fa1818a8188c9365614e6879cccd0e03576ee51c8699534...

000 02 40 20 68 00 64 00 04 00 12 2a 00 7f a1 81 8a .@ h.d.. ..*....
010 81 88 c9 36 56 14 e6 87 9c cc d0 e0 35 76 ee 51 ...6V... ..5v.Q
020 c8 69 95 34 6e ff 9c ec a2 42 cc e1 aa 0e d2 8c .i.4n... .B.....
030 59 02 11 a3 af 3d 9e 67 d2 36 f6 40 bf 12 64 5d Y....=.g .6.@..d]
040 af 17 ef 76 99 c0 ec c4 16 57 45 10 2f 13 e4 25 ...v.... .WE./...%
050 64 ed 91 33 6e f1 e1 0e 15 b8 74 08 d1 10 81 b0 d..3n... ..t.....
060 a9 61 e1 00 9d a9 da 8d b8 75 c9 63 80 .a..... .u.c.



Secret data

- What does this secret data mean?
- 7fa1818a8188c9365614e6879cccd0e03576ee51c86995346eff9ceca242cce1aa0ed28c590211a3af3d9e67d236f640bf12645daf17ef7699c0ecc4165745102f13e42564ed91336ef1e10e15b87408d11081b0a961e1009da9da8db875c9638000000094000000

Reverse



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

```
g.a
transfertype
util
  EncryptCFB$EncryptCfbFailException
  a
  b
  c
  d
  e
    nt : String
    decrypt(byte[], byte[], byte[]) :
    encrypt(byte[], byte[], byte[]) :
    tN(byte[], byte[]) : byte[]
    tO(byte[], byte[]) : byte[]
    tP() : byte[]
    tQ(byte[], byte[]) : byte[]
    tR(byte[], byte[], byte[]) : void
    tS(String, String, String) : byte[]
    tT() : String
    tU(byte[], int) : byte[]
    tV(byte[], int) : byte[]
  AuthService
  NearbyService
  R
  a
  b
  c
  d
  e
  f
  g
  h
  i
```

```
79 private static void tR(byte[] bArr, byte[] bArr2, byte[] bArr3) {
80     byte[] tS = tS(" ", " ", tT());
81     byte[] tL = c.tL(bArr, tS);
82     b.arraycopy(tL, 0, bArr3, 0, 16);
83     b.arraycopy(tL, 16, bArr2, 0, 16);
84     Arrays.fill(tS, (byte) 0);
85 }
86
95 public static byte[] tO(byte[] bArr, byte[] bArr2) {
96     byte[] bArr3 = new byte[16];
97     byte[] bArr4 = new byte[16];
98     tR(bArr2, bArr4, bArr3);
99     return encrypt(bArr, bArr3, bArr4);
100 }
101
109 public static byte[] tN(byte[] bArr, byte[] bArr2) {
110     byte[] bArr3 = new byte[16];
111     byte[] bArr4 = new byte[16];
112     tR(bArr2, bArr4, bArr3);
113     return decrypt(bArr, bArr3, bArr4);
114 }
115
125 private static byte[] tS(String str, String str2, String str3) {
126     if (str == null || str2 == null || str3 == null) {
127         d.e("EncryptCFB", "rootKey is null!");
126         return null;
127     }
128     byte[] decode = Base64.decode(str, 0);
129     byte[] decode2 = Base64.decode(str2, 0);
130     byte[] decode3 = Base64.decode(str3, 0);
131     if (decode.length >= 16 && decode2.length >= 16 && decode3.length >= 16) {
132         return tQ(tV(tQ(tU(decode, 3), decode2), 5), decode3);
133     }
134     d.e("EncryptCFB", "generateRootKey root key is invalid value");
135     throw new EncryptCFB$EncryptCfbFailException("invalid value");
136 }
```




```
private String decryptP2PInfo(String p2pinfo) {
    if (p2pinfo == null) {
        MagicLinkLog.e("MagicLink-BtAndWifiWrapper", "decryptP2PInfo P2Pinfo is null.");
        return null;
    }
    String[] sourceStrArray = p2pinfo.split(":", 3);
    String p2pCfgEx = null;
    if (sourceStrArray.length != 3) {
        MagicLinkLog.e("MagicLink-BtAndWifiWrapper", "decryptP2PInfo P2Pinfo is invalid.");
        return null;
    }
    try {
        p2pCfgEx = Encrypt.decrypt(sourceStrArray[0], Encrypt.generateSecureKey(Encrypt.g
        " ", " ", this.mSecureRandom), sourceStrArray[2]
    } catch (Exception e) {
        e.printStackTrace();
    }
    return p2pCfgEx;
}
```




Older version

```
public static String decrypt(String sSrc, String sKey, String slv) {  
    if (sSrc == null || sKey == null || slv == null) {  
        MagicLinkLog.e("BleEncrypt", "decrypt Key is null");  
        return null;  
    } else if (sKey.length() != 25) {  
        MagicLinkLog.e("BleEncrypt", "decrypt - Length of Key is not 25byte" + sKey.length());  
        return null;  
    } else {  
        byte[] byteKey = Base64.decode(sKey, 0);  
        byte[] bytelv = Base64.decode(slv, 0);  
        byte[] encrypted = Base64.decode(sSrc, 0);  
        SecretKeySpec sKeySpec = new SecretKeySpec(byteKey, "AES");  
        try {  
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");  
            cipher.init(2, sKeySpec, new IvParameterSpec(bytelv));  
            return new String(cipher.doFinal(encrypted), "UTF-8");  
        } catch (RuntimeException e) {  
            e.printStackTrace();  
            return null;  
        } catch (Exception e2) {  
            e2.printStackTrace();  
            return null;  
        }  
    }  
}
```




Decrypt

```
p2pCfgEx = Encrypt.decrypt(sourceStrArray[0],  
Encrypt.generateSecureKey(Encrypt.generateRootKey("XXXXXXXXXXXXXXXXXX", "  
XXXXXXXXXXXXXXXXXX", this.mSecureRandom), sourceStrArray[2]), sourceStrArray[1]);
```

```
this.mSecureRandom =  
service.getApplicationContext().getResources().getString(R.string.ble_xxxx_key);
```

/res/values/strings.xml:

```
<string name="ble_xxxx_key">XXXXXXXXXXXXXXXXXX</string>
```




Decrypt

- Get **sourceStrArray** from BLE packet
- **sSrc** = sourceStrArray[0]
- **slv** = sourceStrArray[1]
- **Rootkey** = enerateRootKey(" **XXXXXXXXXXXXXXXXXXXX** ",
" **XXXXXXXXXXXXXXXXXXXX** ",mSecureRandom)
- **sKey** = generateSecureKey(rootkey,sourceStrArray[2])
- **p2p_info** = decrypt(sSrc,sKey,slv)

Decrypt



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

```
$ python extract_password.py ble.pcap
rootkey: 
sKey hex: 
sKey: 
sSrc: cBPZzu0AoBJWUJ9/rqT0FsBYCHtBoGDMWGVbgiI+G5M08CwKXhhol2ZvIIceClnV+g0C30CaqKt+\nzUgWN2klMpEHmldlmhSRrrxicFaLBNQ=\n
sIv: 8HqTPrzd9nuBzoMkV08/g==\n
P2Pinfo:
```

```
=====
id: ('ygtat', 'BBBB')
ip: 192.168.49.35
ssid: DIRECT-dZ-
password: xQTpdRN2
version: 2462##versio
=====
```

```
$ python new_extract_password.py ble.pcap
data 7fa1818a8188c9365614e6879cccd0e03576ee51c86995346eff9ceca242cce1aa0ed28c590
211a3af3d9e67d236f640bf12645daf17ef7699c0ecc4165745102f13e42564ed91336ef1e10e15b
87408d11081b0a961e1009da9da8db875c9638000000094000000
len: 208
Got p2p information!
P2Pinfo:
```

```
=====
ip: 192.168.49.2
ssid: DIRECT-ow-AAAA
password: DRtkSDs1
version: 2412##version:3#
=====
```




Sniffer

- Sniffer BLE
- Extract the secret data
- Decrypt the secret data
- Join the P2P Group
- ARP spoofing
- **Got the secret files!**

Demo 1



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Demo 2



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Man-in-the-middle attack



Device and ID spoofing

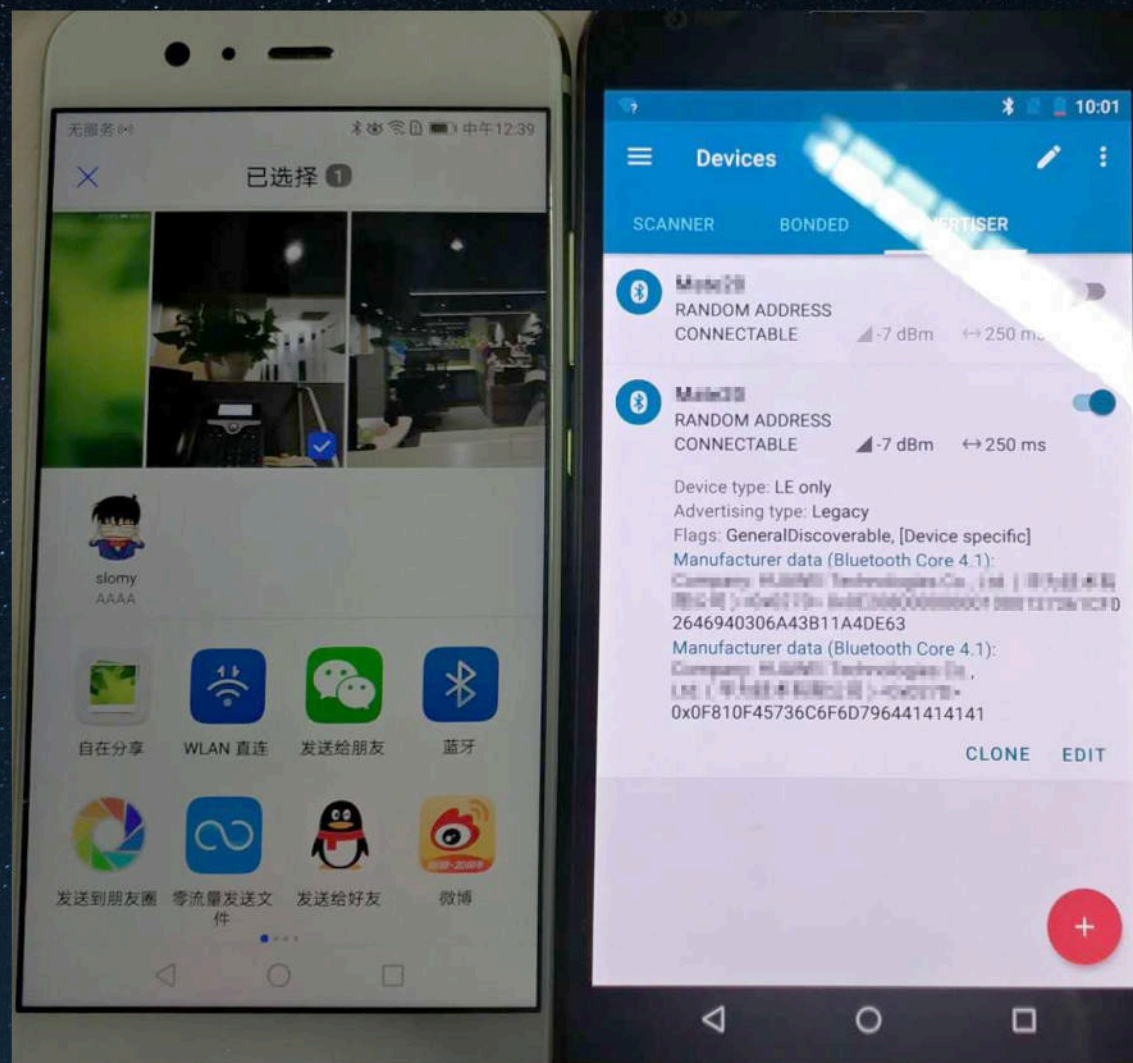
- Who is **herbo zhang**?
- How to confirm identity?
- **DEVICE? ID? ICON?**



Device and ID spoofing



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

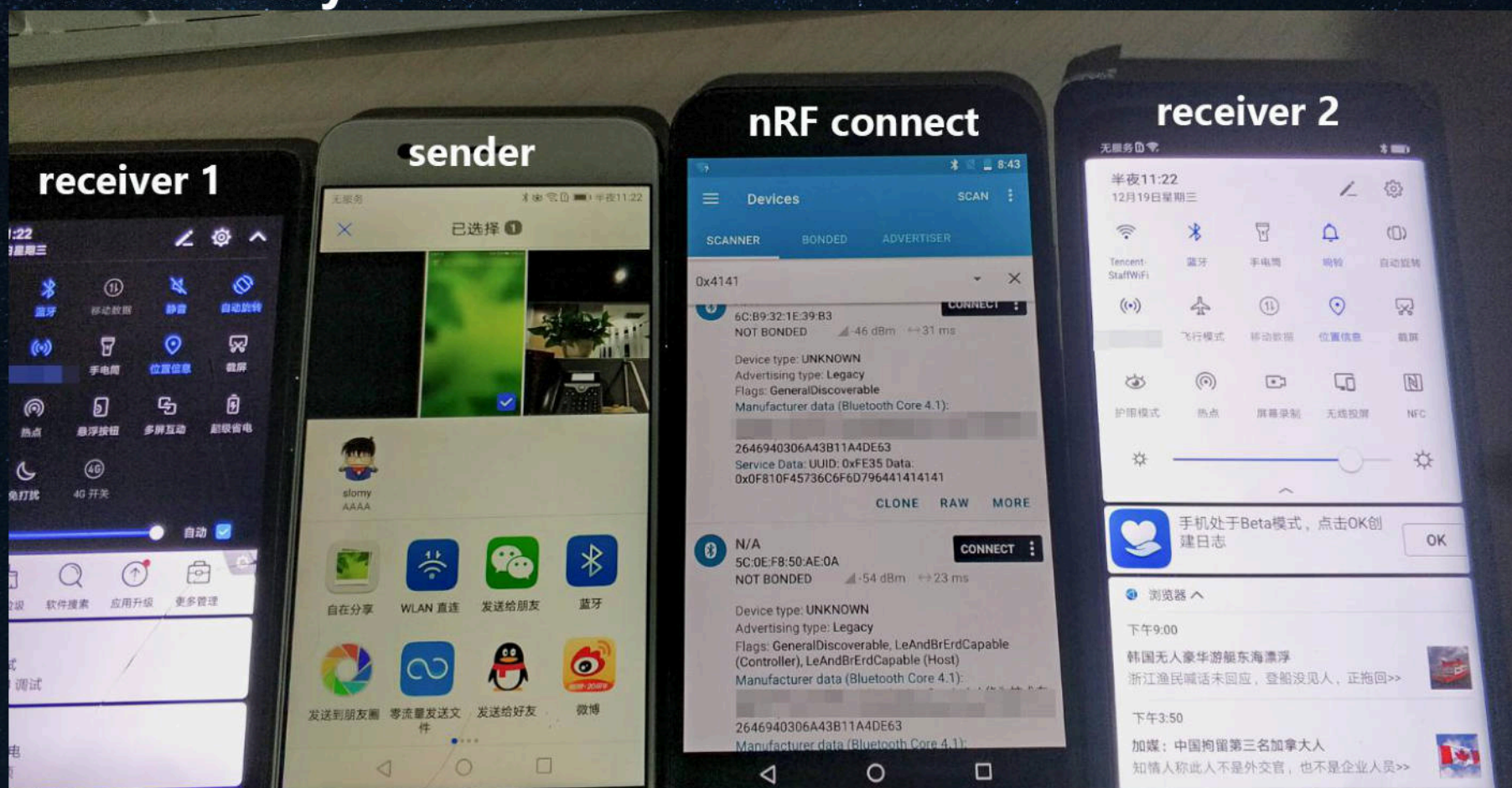


Device and spoofing



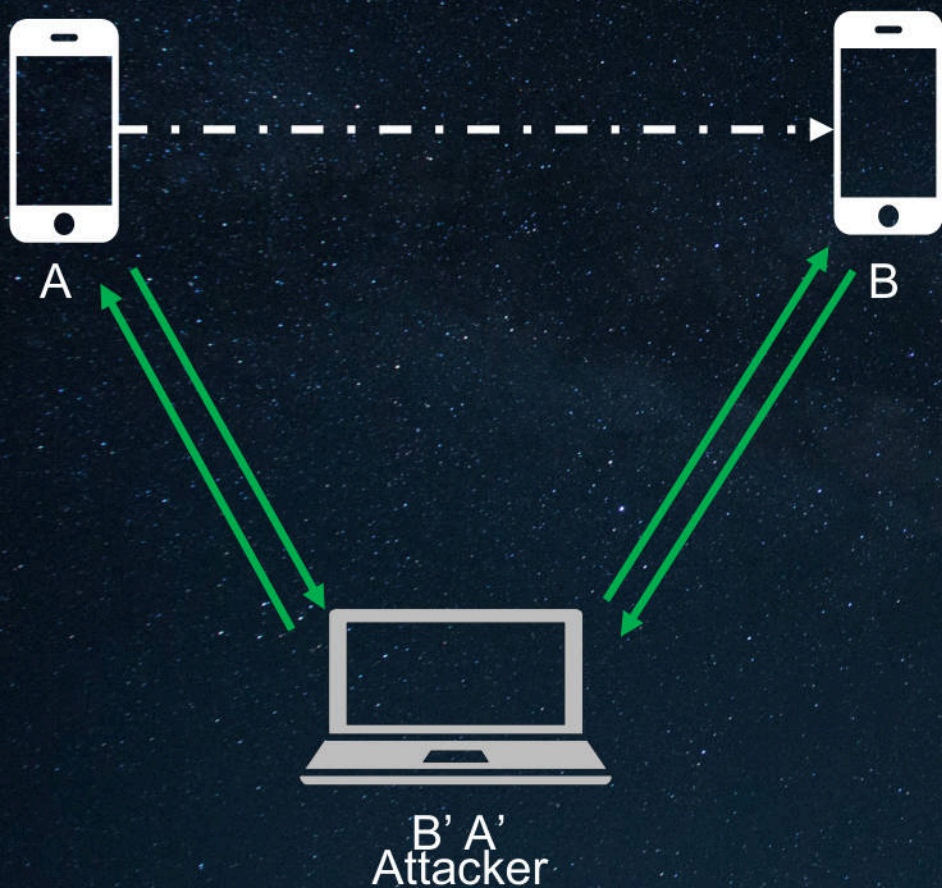
腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

- Who is really AAAA?





Man-in-the-middle



Who is the real receiver?

A has 50% chance to choose attacker.



Man-in-the-middle

- A can't find the attacker A'
 - The attacker does not send a scan response to A
- B can't find the attack B'
 - B is not in discovery mode
- Attacker can distinguish between A and A', B and B'
 - The attacker can choose to block or receive any message
- The users (A,B) can't feel anything

Demo 3



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Logic vulnerabilities

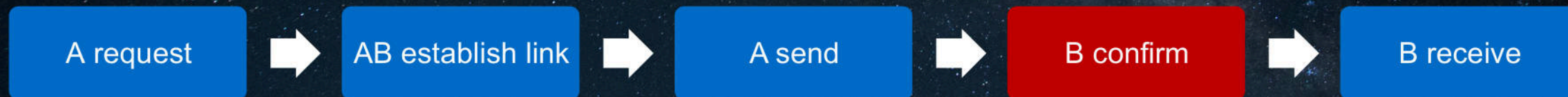


None-confirm connection

- Secure connection process



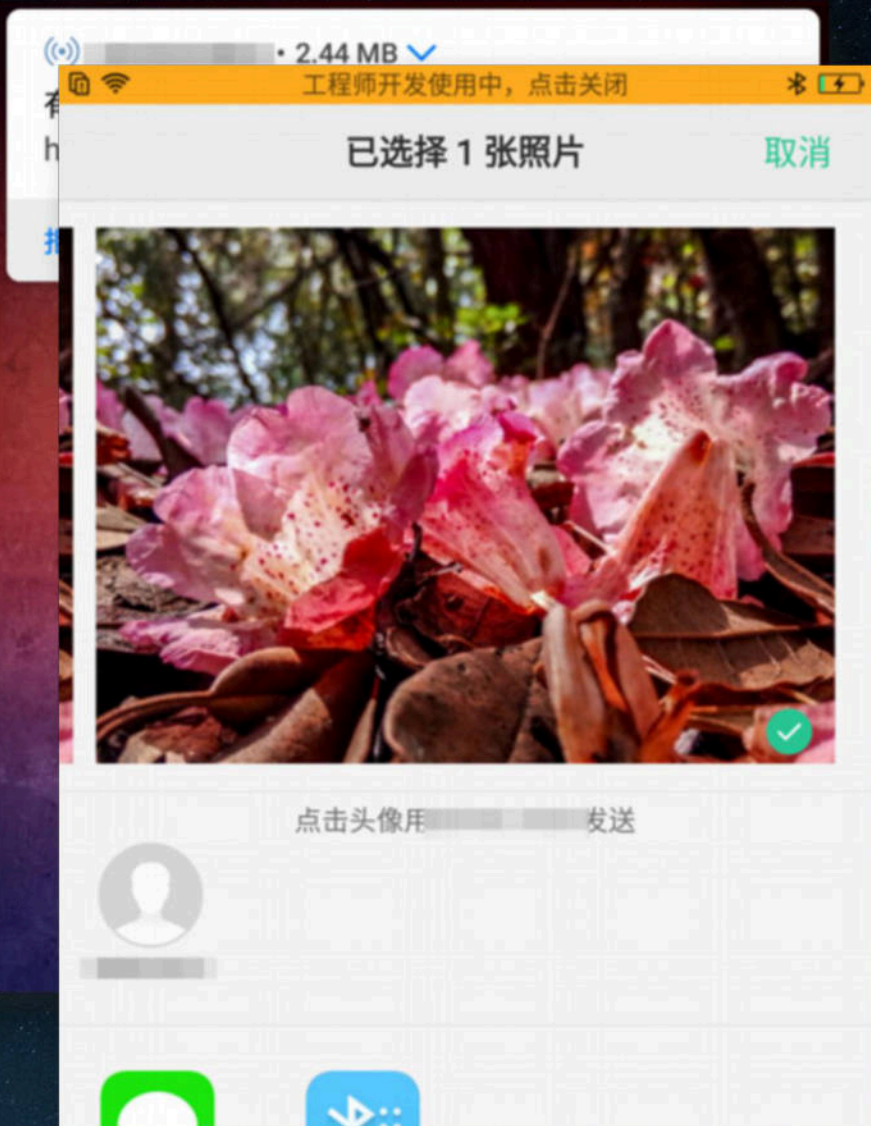
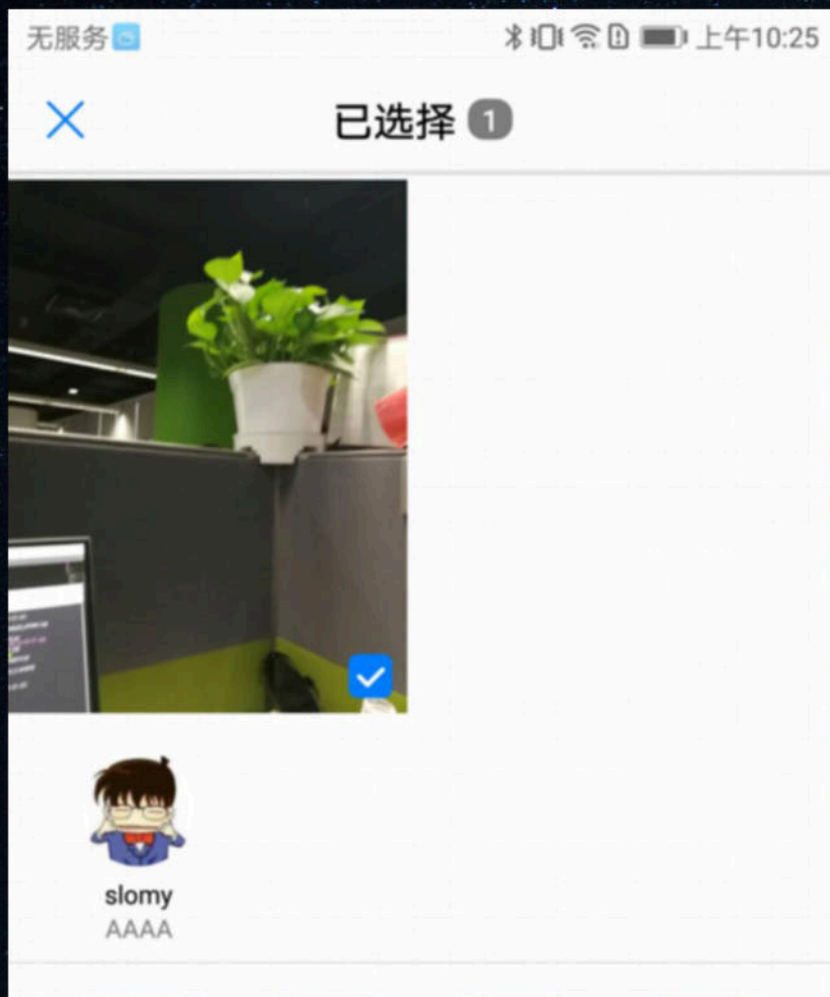
- Unsecure connection process



None-confirm connection



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB





None-confirm connection

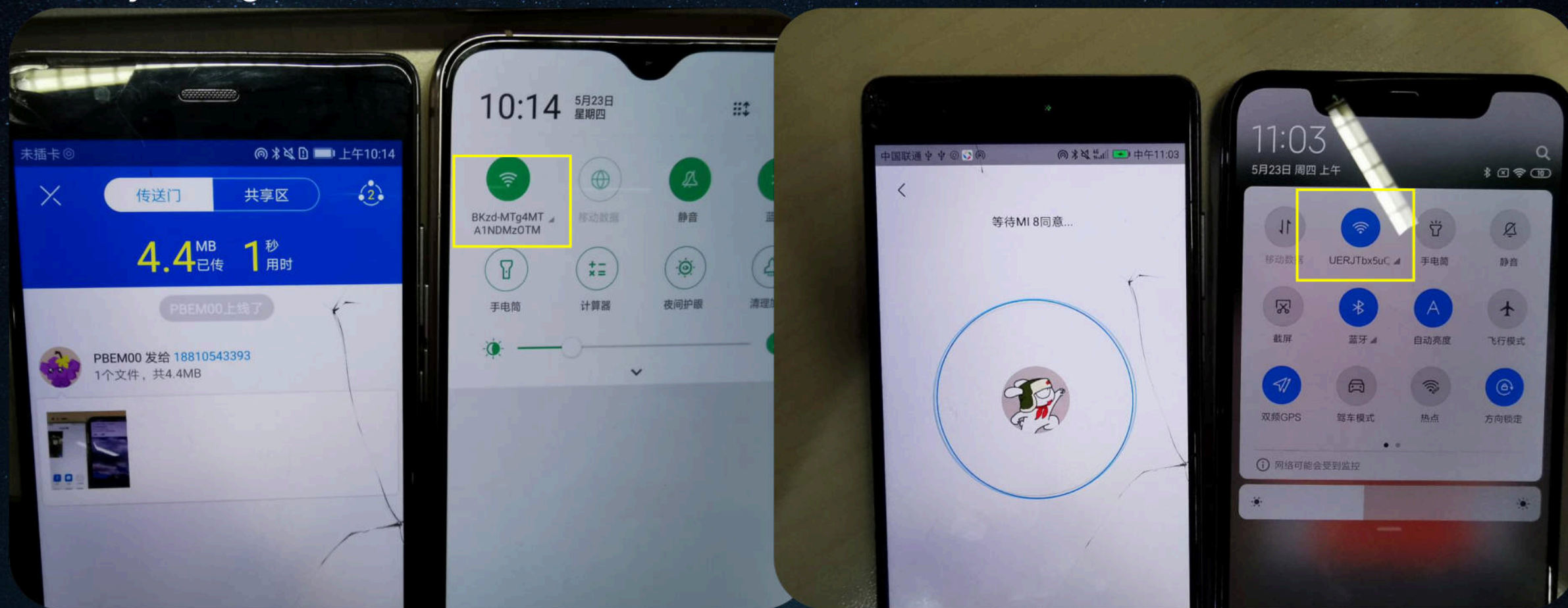
- Bring more attack surfaces

Zero click exploit?



None-confirm connection

- Hijacking the network





Hijacking the network

- Who is server? Sender? Receiver?
- Can I choose to be a server? Yes!
- Can I make the other user connect to my WiFi without interaction? Yes!

Demo 4



腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB



Other vulnerabilities



Other vulnerabilities

- Accept automatically via Wi-Fi P2P
- Directory traversal
- RFM Vulnerabilities



Wi-Fi/Wi-Fi P2P Access Control

- Need to input password





Accept automatically via Wi-Fi P2P

- However, when we use Wi-Fi P2P protocol to connect:
 - All authentications are gone!





Directory traversal

- A: Can I send you a file?
- B: Yes
- B: Please send it. I'm ready.
- A: I want to send it to you: /PATH/AAAA.JPG
- B: Send GET request for /PATH/AAAA.JPG
- A: Send /PATH/AAAA.JPG

Demo 5

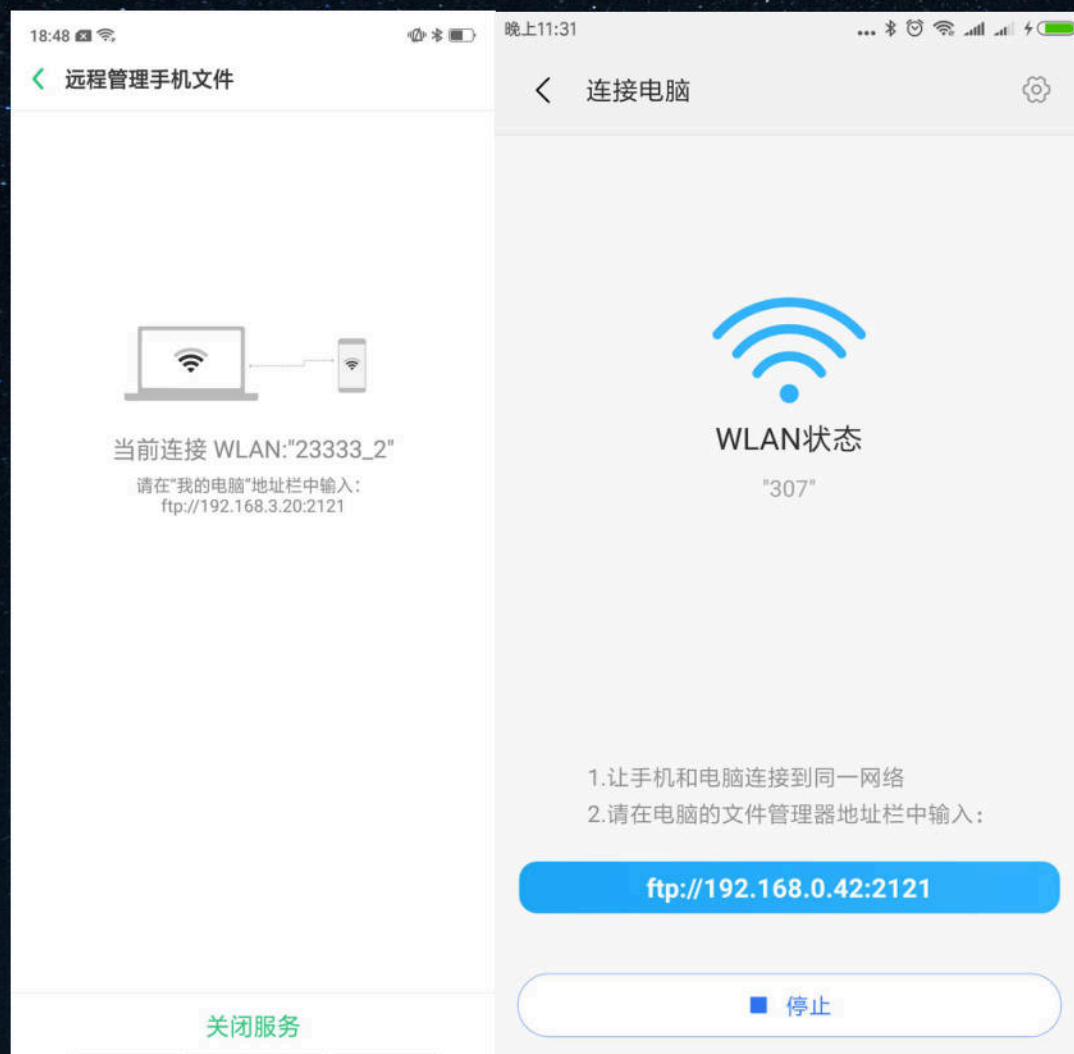


腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB

Remote file management on computer



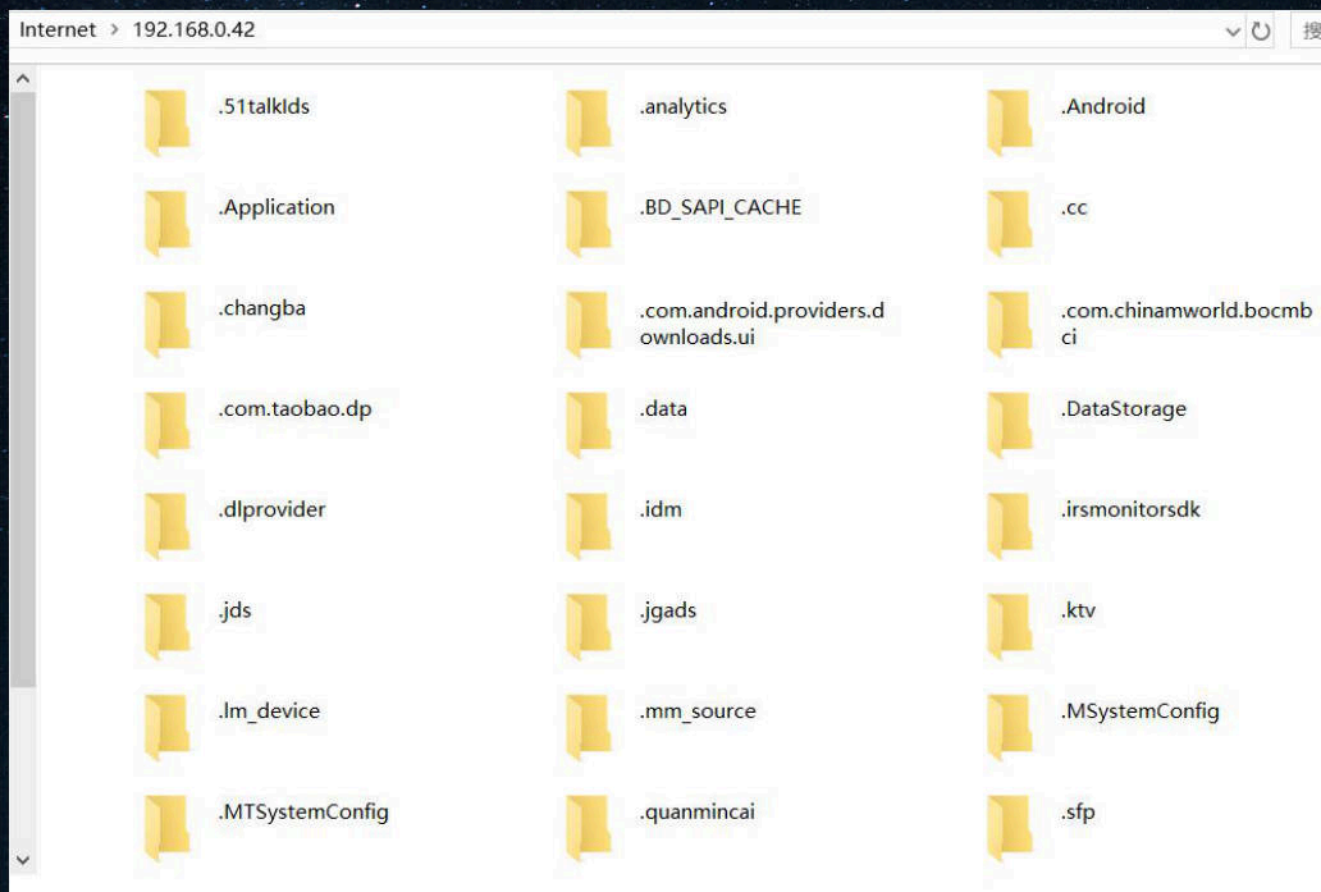
腾讯安全玄武实验室
TENCENT SECURITY XUANWU LAB





RFM Vulnerabilities

- Anonymous users have **READ** and **WRITE** permissions





Summary

- None-confirm before WiFi Connection Established
- Not well protected access to WiFi/WiFi P2P
- Unsecure Transport
- No anti-spoofing

The background of the slide is a dark, deep blue night sky filled with numerous small, bright white stars. A faint, wispy band of light, resembling the Milky Way, stretches across the lower right portion of the image. The overall effect is a cosmic and mysterious atmosphere.

Nearby Sharing Apps Security Survey



Pre-installed Nearby Sharing Apps

App	Confirm before WiFi Connection Established	WiFi/WiFi P2P Authentication	Secure Transport	Prevent Spoofing
A	×	○	×	×
B	×	○	×	×
C	×	○	×	×
D	×	○	×	×

× : No security measures

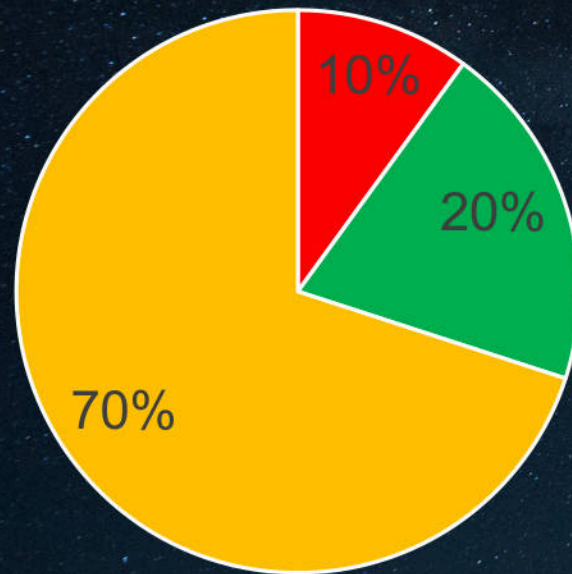
○ : Have security measures but can be bypassed

✓ : Have security measures

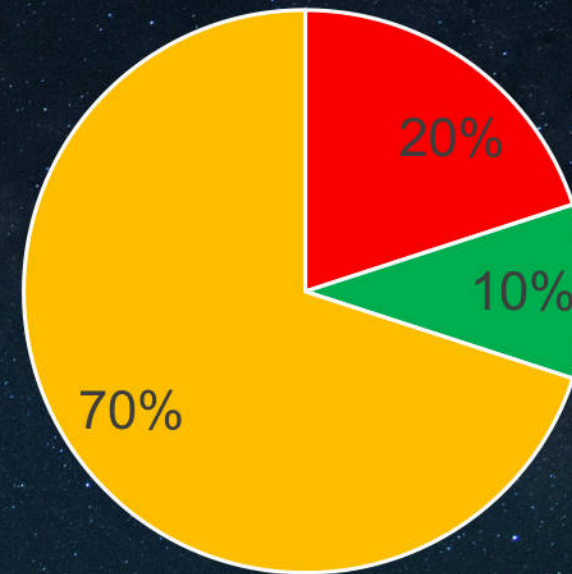


Top 10 Third Party Nearby Sharing Apps

Sniffer Attack



Spoofing



■ : Vulnerable
■ : Safe
■ : Partial Vulnerable

Mitigation



Mitigation

- More secure Wi-Fi / Wi-Fi P2P Key Exchange
- Transport encryption
- Prevent spoofing
- Others tips



Secure WiFi/WiFi P2P Key Exchange

- Transfer the KEY over a secure channel
 - Out of band: NFC
 - Certificate based key-exchange mechanism
 - Transferring keys over the Internet securely
- PIN / PBC



Secure Transport

- Using TLS/HTTPS instead of TCP/HTTP
- Transferring keys over the Internet securely
- Pre-exchanged keys



Prevent spoofing

- Certificate based authentication
 - Unique ID
 - Distributed by the server
 - Signed with pre-defined trust anchor
- Attacker can't sign the Unique ID correctly
- Make sure that the device can distinguish between real users and attackers

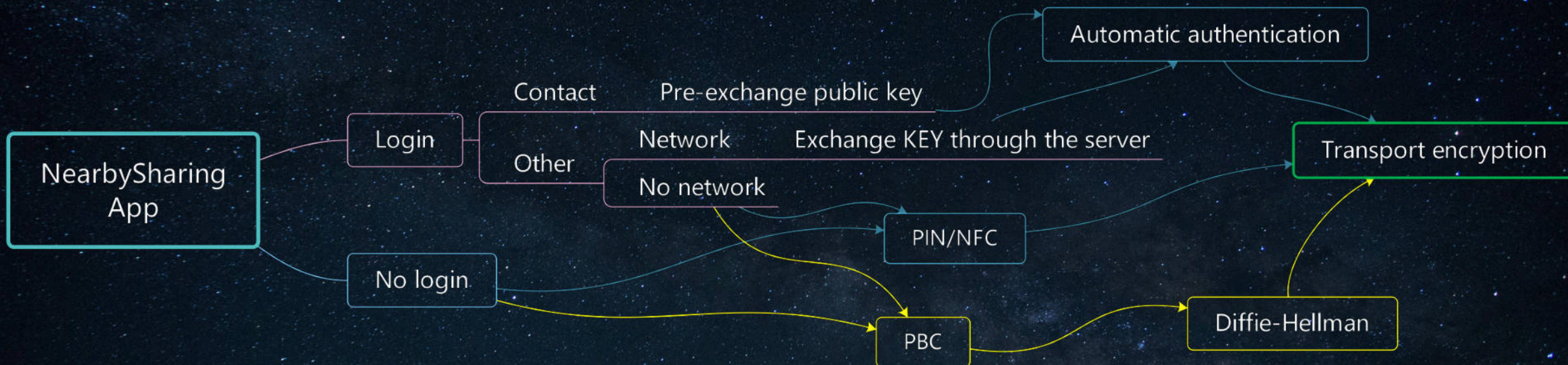


Others tips

- Prevent attackers from joining the network
- Turned off by default, turned off after a period of idle time
- Confirm before connection established
- **Do not establish connection automatically with users who are unidentifiable**



Recommended design





Conclusion

- Attack vectors of nearby sharing apps
- First comprehensive research about them
- Found Many vulnerabilities categorized in several common attack methods
- Present some effective mitigation methods

Q&A
